

Week 6: Stack and Heap Memory in a Multithreaded Recursive Program

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

Aug 16th, 2026

Part One: Memory Diagram

`fibonacci(int n)` returns 0 for $n = 0$, 1 for $n = 1$, and otherwise `fibonacci(n-1) + fibonacci(n-2)`. With $n = 2$ it makes two child calls, `fibonacci(1)` and `fibonacci(0)`, evaluated one at a time and summed to 1, so the recursion reaches only two frames above `Worker.run()`. `main()` builds three `Worker` objects (`threadId, n = 2`), wraps each in a `Thread`, starts all three, and joins them. Figure 1 shows memory at the moment each thread sits at its deepest frame.

JVM Memory Layout: Three Threads Executing `fibonacci(2)`

SampleFibonacci.java — per-thread call stacks, shared heap allocation, and a shared static counter

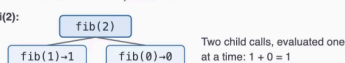
Recursive function (depth 2 for $n = 2$)

```
static int fibonacci(int n) {
    if (n == 0) return 0; // base case
    if (n == 1) return 1; // base case
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Multithreaded scenario

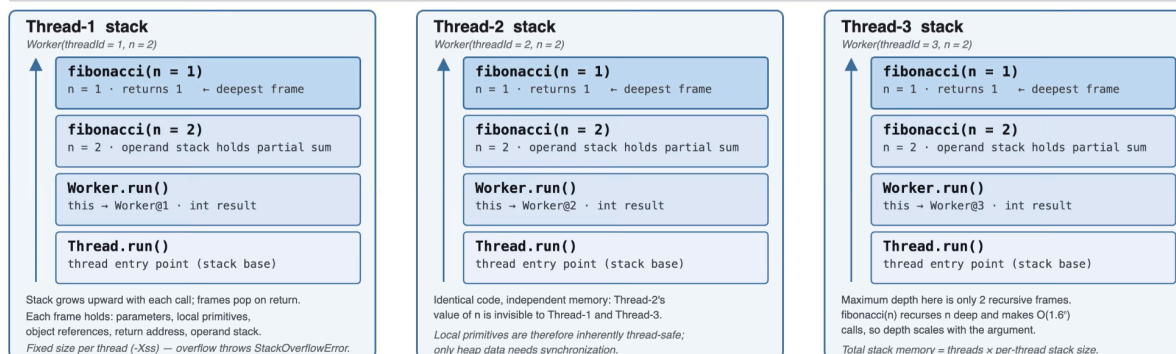
`main()` launches `Thread-1`, `Thread-2`, `Thread-3` — each wraps a `Worker(threadId, n = 2)`. Every thread runs the same code but on its own private stack.

Call tree for `fibonacci(2)`:



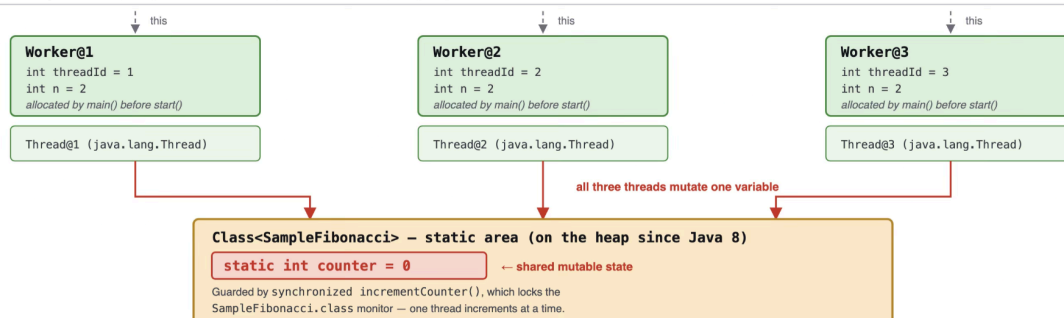
1 Per-Thread JVM Stacks

private to each thread — never shared, reclaimed automatically when a frame returns



2 Java Heap

shared by every thread — where all objects live, reclaimed by the garbage collector



3 Metaspace

native memory outside the heap (replaced PermGen in Java 8)

Class metadata: bytecode for `fibonacci()` and `Worker.run()`, method tables, field layouts, runtime constant pool

Holds the definition of each class. The static field's value lives with the Class object on the heap above, not here.

Stack frame (per-thread) Heap object (shared) Static area (shared) Metaspace (native) - - - - Object reference - - - - Contended write (race risk)

Figure 1. JVM memory during three concurrent fibonacci(2) calls. Dashed arrows are references from a stack into the heap; red arrows mark the single contended write.

Part Two: Reflection

Role of each labeled region

Each thread owns a private stack (region 1) with one frame per invocation, holding that call's parameters, local primitives, object references, return address, and operand stack. Because `n` and result are primitives they live entirely in the frame, which is destroyed on return without collector involvement. The heap (region 2) is shared and holds every object — three `Worker` and three `Thread` instances — and each `Worker.run()` frame's `this` reference points from a private stack into it. The static area within the heap holds counter: one copy per class rather than per instance, stored with the `Class` object since Java 8. Metaspace (region 3) is native memory outside the heap holding class metadata such as bytecode and method tables.

Why constraining stack growth matters

Each stack is a fixed reservation (`-Xss`, commonly 512 KB to 1 MB), which bounds recursion depth: `fibonacci(n)` recurses `n` deep, and a large `n` throws `StackOverflowError`, which cannot be handled meaningfully. This function's cost is exponential in calls but only linear in depth, so it is `n`, not call count, that threatens the stack. At scale the sharper limit is that total stack memory is threads times stack size — a thread-per-request server with 1,000 threads reserves roughly a gigabyte before doing any work — which is why production systems bound recursion and use thread pools.

A concurrency risk revealed by the diagram

Three private stacks converge on one shared cell: `static int counter`. Everything above it is thread-confined, so `counter` is the only contended location. `counter++` is a read-modify-write, and interleaving two of them loses an update; `synchronized incrementCounter()` prevents exactly that. The diagram exposes a subtler defect, though: `run()` reads `counter` again in its `printf` call, outside the lock. That read has no happens-before relationship with other threads' increments, so it may see a stale value, and another thread can increment between this thread's increment and its read. The increment is atomic; increment-then-report is not.

A synchronization technique that solves it

Widen the critical section so the increment and the read of its result are one atomic step. The minimal change returns the new value from inside the lock:

```
public static synchronized int incrementCounter() {  
  
    return ++counter;  
  
}
```

`run()` then prints that returned `int` — a local in its own frame, and therefore thread-confined.

`AtomicInteger.getAndIncrement()` is an alternative, performing the same read-modify-write as one lock-free compare-and-swap and scaling better because threads never block. Either way, the unit of atomicity must cover the whole invariant, read together with write, not the write alone.

Acknowledgment of AI Use

To complete this activity, I utilized a generative AI tool—Claude Code (powered by Anthropic's Claude Opus 5).